

Technical Architecture Document: SecureCare CMS

1. Architectural Pattern

The solution adheres to **Clean Architecture (Onion Architecture)** to ensure separation of concerns, testability, and independence from external frameworks.

Layered Structure

- **Core (Domain Layer)**
 - *Role:* The heart of the software. Contains enterprise logic and types.
 - *Contents:* Entities, Value Objects, Domain Events, Repository Interfaces, Domain Services.
 - *Dependencies:* None.
- **Application Layer**
 - *Role:* Orchestrates application flow and business rules.
 - *Contents:* CQRS Handlers (Commands/Queries), DTOs, Validators, AutoMapper Profiles.
 - *Dependencies:* Project Reference to Core.
- **Infrastructure Layer**
 - *Role:* Implements interfaces defined in Core; interacts with external concerns.
 - *Contents:* Entity Framework DB Context, Migrations, Repository Implementations, Email Service, Auth Service (JWT Generation).
 - *Dependencies:* Project Reference to Application and Core.
- **API Layer (Presentation)**
 - *Role:* Entry point for the application.
 - *Contents:* REST Controllers, Middleware (Global Error Handling), Dependency Injection Setup.
 - *Dependencies:* Project Reference to Application and Infrastructure.

2. Design Patterns & Principles

- **CQRS (Command Query Responsibility Segregation):**
 - Reads (Queries) are separated from Writes (Commands).
 - Library: **MediatR**.
- **Repository Pattern:**
 - Abstractions in Core (e.g., IPatientRepository).
 - Implementations in Infrastructure (e.g., PatientRepository).
- **Unit of Work:**
 - To manage atomic transactions across multiple repositories.
- **Dependency Injection:**
 - Built-in .NET Core DI container.

3. Data Model (ERD)

Entities

User

Base entity for authentication. * Id (GUID, PK) * Email (String, Unique) * PasswordHash (String) * Role (Enum: Admin, Doctor, Receptionist, Patient) * IsActive (Boolean)

Doctor

Extends User. * Id (GUID, PK, FK -> User.Id) * LicenseNumber (String) * Specialization (String)

Patient

Extends User. * Id (GUID, PK, FK -> User.Id) * NationalID (String, Unique) * DateOfBirth (Date) * Gender (String) * Phone (String) * EmergencyContact (String)

Appointment

aggregate Root * Id (GUID, PK) * PatientId (GUID, FK -> Patient) * DoctorId (GUID, FK -> Doctor) * DateTime (DateTimeOffset) * Status (Enum: Pending, Confirmed, Completed, Cancelled)

Prescription

Immutable Record * Id (GUID, PK) * AppointmentId (GUID, FK -> Appointment) * MedicationDetails (Text) * Notes (Text) * CreatedDate (DateTime, Immutable)

4. Security Architecture

Authentication

- **Mechanism:** JWT (JSON Web Tokens).
- **Flow:** Client exchanges credentials for a Bearer Token.
- **Token Claims:** sub (UserId), email, role, jti (TokenId).

Authorization

- **Policy-Based Authorization:**
 - Policies defined in Program.cs rather than hardcoded roles on controllers.
 - *Example:* Policy("RequireClinician") checks for Role = Doctor.
- **Resource-Based Authorization:**
 - Handlers must check if the authenticated user owns the data (e.g., Patient viewing own records).

5. Technology Stack

- **Platform:** .NET 10
- **Web Framework:** ASP.NET Core Web API

- **Language:** C# 12
- **Database:** PostgreSQL / SQL Server
- **ORM:** Entity Framework Core
- **Validation:** FluentValidation
- **Testing:** xUnit, Moq, FluentAssertions